

Full Text Search on SQL Server Part 3

By Don Schlichting

In the previous month's articles, Full Text Searching was introduced as a way to query strings with more refinement than the usual TSQL "like" or equal operator statements. With Full Text Searching, a new file system structure is created, storing key words from selected fields into Catalogs. In addition to storing typical character fields from databases, Microsoft Office documents that have been saved as binaries can also be entered into the Catalogs. In this month's edition, we will begin with the maintenance issues required to keep these Catalogs current. Once created, these catalogs can be interrogated for key words that are near each other's proximity, the singular and plural versions of a word, or the noun and verb variations of a word. In addition, search result rankings, or weights, that usually accompany internet search engine returns can also be requested.

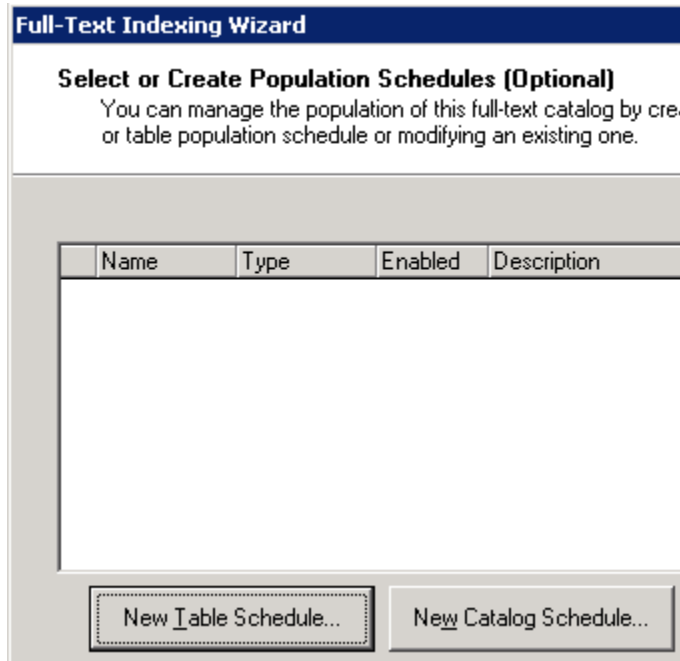
Population Schedules

By default, a Full Text Search Catalog is not kept in sync with its underlying table source. We will start this month's article by exploring the options available for Catalog synchronization, also known as populating. Let's begin by creating a new catalog.

- 1.) From Enterprise Manager, select the pubs database, then from the top menu, select tools, Full Text Indexing. The full text wizard should appear. If the full text options are grayed out, either the search service is not installed, or you are on a non-server OS. Running Enterprise manager from a workstation and registering a server will not work. Select Next from the wizard.
- 2.) Select the authors table and click Next.
- 3.) Select the default unique index and click next. Full text indexing only supports tables with a unique index.
- 4.) On the columns screen, select first name, last name, and city, then click Next.
- 5.) If you have a previously created Catalog, you will now be prompted to include this new search into the existing Catalog, or to create a new one. For this example, a new Catalog will be created called demo2. Then click Next.

We should now be on the "Select or Create Population Schedules" screen. In the first article of this series, we exercised the option of not having a population schedule. Population scheduling is not a requirement. In the Part 1 example, we created an empty Catalog, then, after creation, we did a one time, manual population. In that example, the Catalog would never receive another update, unless it was done manually again. This would be a good option if the database was closed to DML and only a one-time population was needed. In the majority of cases, the database will be receiving updates, changes, and deletions, and we will want these reflected in the Catalog. There are three major methods of keeping Catalogs current, Full Rebuilds, Incremental Rebuilds using a timestamp, and Change Tracking, where each change to the source table is logged. This last option can work like a trigger, where DML operations on the underlying table cause the Catalog to update in the background. This has the benefit of keeping the Catalog always in sync with the underlying database table in almost real time. However, that freshness comes at the

cost of increased CPU utilization. Although a Change Track update can be scheduled, it is usually selected for its ability to do background updates. The two other options, Full and Incremental, use a scheduling feature, which will be presented in this screen. (Implementing a Background Update is not done from these screens; we will cover it shortly.)



Full-Text Indexing Wizard

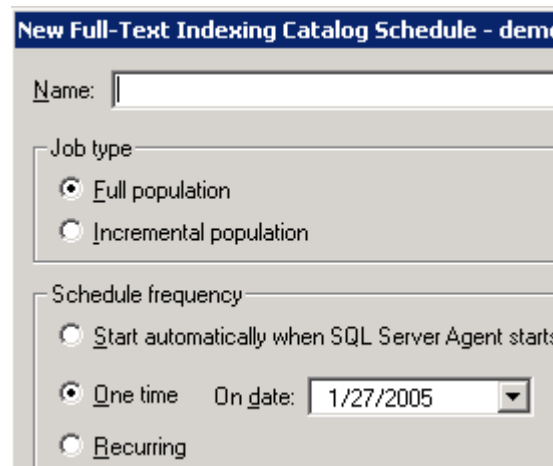
Select or Create Population Schedules (Optional)

You can manage the population of this full-text catalog by creating a new table population schedule or modifying an existing one.

Name	Type	Enabled	Description
------	------	---------	-------------

New Table Schedule... New Catalog Schedule...

The first choice here is to schedule updates by table, or by Catalog. Recall from the previous step, we had the option of including our table selection into an existing Catalog. In that case, we could opt to update the entire Catalog at once, rather than separate jobs for each table. Clicking the "New Catalog Schedule" gives the options of Full or Incremental populations.



New Full-Text Indexing Catalog Schedule - demo

Name:

Job type

☒ Full population

☐ Incremental population

Schedule frequency

☐ Start automatically when SQL Server Agent starts

☒ One time On date:

☐ Recurring

A Full population will completely repopulate every index included in the Catalog. (When a table is selected for inclusion in Full Text, an index is created. See the previous articles for

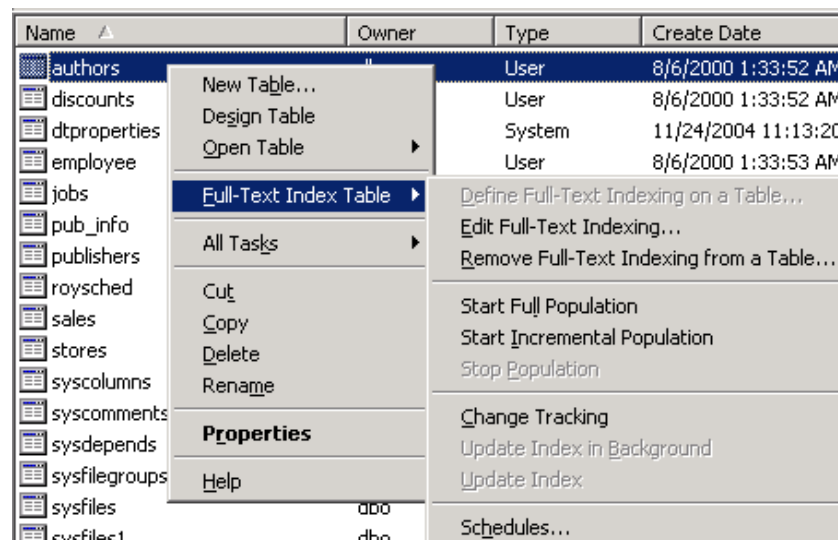
additional information.) Think of this as doing a DELETE * followed by an INSERT of all records. If the number of rows in the Catalog is very large, this may take considerable time. However, it can be scheduled for an off peak hour using the Scheduling Frequency selects. Incremental Populations only update the index with any changes (insert, update, delete) that took place on the underlying table since the last population. This option requires the underlying table to contain a timestamp field. If you are coming to MS SQL from another database, you may be familiar with a timestamp being a date time combination field. This is not the case in TSQL. A TSQL timestamp is an eight-byte, unique, auto generated binary number. Its only purpose is version control. (See "rowversion data type" in BOL for additional details.) This timestamp is used as the control for Incremental populations. (Note: if you are using the WRITETEXT or UPDATETEXT methods, the timestamp field is NOT changed.)

Table Schedules give two of the same options, Full and Incremental, that were just seen in the Catalog selects. They work the same at the table level as they do at the Catalog level. The third option, Update Index, can be used with Change Tracking, which we will come back to. If the option of Incremental is selected, and there is not a timestamp field in the underlying table, a Full Update will be used.

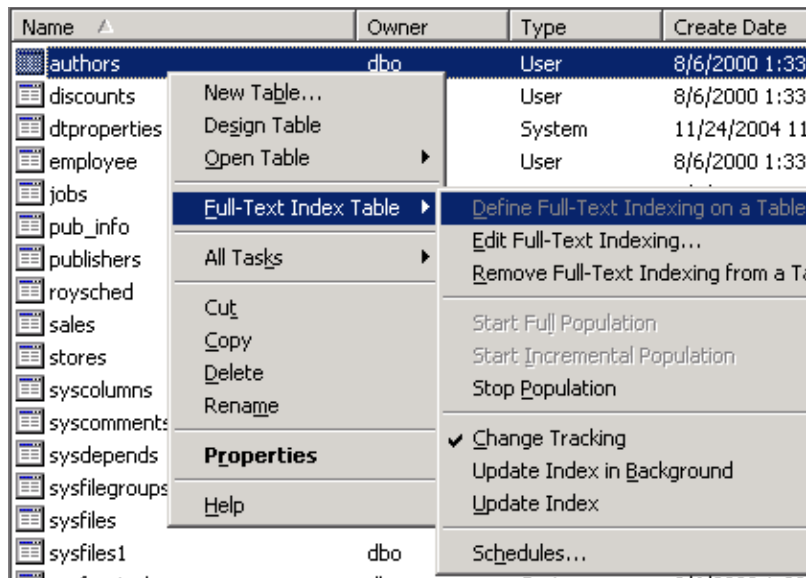
For this example, click cancel, so no schedule is created. Click Next, then Finish. The wizard should create a new Catalog based on the authors table.

Change Tracking

Change Tracking is a population method that creates a hidden "change record" every time there is a DML operation on the underlying table. These "change records" can update the Catalog immediately, or be scheduled (Update Index). If they are applied immediately, then the Catalog is up to date in near real time status with the underlying table. To enable Change Tracking, right click the authors table in Enterprise Manager. Next, select Change Tracking.



Right click authors once more, and now that Change Tracking was previously invoked, select Update Index in Background. Now, every time a table change takes place, the Catalog will be immediately updated.



Conclusion

There are several different methods available to keep Catalogs current. With multiple update types and schedules to choose from, almost any configuration requirement can be accommodated. Next month we will address some additional house keeping tasks and cover searching Microsoft Office documents.

COMMENTS

Thanks to all for the comments and suggestions for this series. I have tried to respond to all with specific questions, but several emails I received where deleted (user error on my part), before I could reply. If your email was one of them, please resend. Thanks.